



LeetCode 70: Climbing Stairs

1. Problem Title & Link

Title: LeetCode 70: Climbing Stairs

Link: <https://leetcode.com/problems/climbing-stairs/>

2. Problem Statement (Short Summary)

You are climbing a staircase with n steps.

Each time you can climb:

- 1 step
- 2 steps

Find the total number of distinct ways to reach the top.

Return the number of possible ways.

3. Examples (Input → Output)

Example 1

Input:

$n = 2$

Output:

2

Explanation:

Possible ways:

$1 + 1$

2

Total:

2 ways

Example 2

Input:

$n = 3$

Output:

3

Explanation:

$1 + 1 + 1$

$1 + 2$

$2 + 1$

Total:



3 ways

Example 3

Input:

$n = 5$

Output:

8

Explanation:

1+1+1+1+1

1+1+1+2

1+1+2+1

1+2+1+1

2+1+1+1

1+2+2

2+1+2

2+2+1

4. Constraints

$1 \leq n \leq 45$

Important:

n is small enough to fit inside an integer.

5. Core Concept (Pattern / Topic)

Dynamic Programming

Secondary Topics:

- Fibonacci Sequence
- Recursion
- Memoization

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

Suppose:

$n = 4$

To reach step 4:

You could come from:

step 3 + 1 step



OR

step 2 + 2 steps

Therefore:

$\text{ways}(4)$

=

$\text{ways}(3) + \text{ways}(2)$

Similarly:

$\text{ways}(5)$

=

$\text{ways}(4) + \text{ways}(3)$

General Formula

For any stair:

$\text{ways}(n)$

=

$\text{ways}(n-1)$

+

$\text{ways}(n-2)$

Because:

Last move is either:

1 step

or

2 steps

Recognizing the Pattern

Let's calculate:

$n=1 \rightarrow 1$

$n=2 \rightarrow 2$

$n=3 \rightarrow 3$

$n=4 \rightarrow 5$

$n=5 \rightarrow 8$



$n=6 \rightarrow 13$

Looks familiar?

1 2 3 5 8 13 ...

This is the Fibonacci sequence.

Why Dynamic Programming?

Recursive solution repeatedly calculates the same values.

Example:

ways(5)

```

      5
     / \
    4   3
   /\  /\
  3 2 2 1

```

Notice:

ways(3)

ways(2)

are calculated multiple times.

Dynamic Programming stores answers and avoids repetition.

7. Visual / Intuition Diagram (ASCII Diagram)

For:

$n = 4$

```

      4
     / \
    3   2
   /\  /\
  2  1 1 0
 /\ 
1  0

```

Each node represents:

ways(current_step)

DP Table:

Step : 1 2 3 4 5

Ways : 1 2 3 5 8



Relationship:

```
ways[5]
```

```
=
```

```
ways[4]
```

```
+
```

```
ways[3]
```

```
8 = 5 + 3
```

8. Pseudocode (Language Independent)

```
if n <= 2
    return n

first = 1
second = 2

for i from 3 to n

    current = first + second

    first = second
    second = current

return second
```

9. Code Implementation

Python

```
class Solution:
    def climbStairs(self, n: int) -> int:

        if n <= 2:
            return n

        first = 1
        second = 2

        for i in range(3, n + 1):

            current = first + second

            first = second
```



```
        second = current

    return second
```

Java

```
class Solution {

    public int climbStairs(int n) {

        if(n <= 2)
            return n;

        int first = 1;
        int second = 2;

        for(int i = 3; i <= n; i++) {

            int current = first + second;

            first = second;
            second = current;
        }

        return second;
    }
}
```

10. Time & Space Complexity

Iterative DP Solution

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Why?

One loop from 3 to n.

Only three variables used.

11. Common Mistakes / Edge Cases

Mistake 1

Wrong Base Cases

Wrong:



```
if n == 1:  
    return 1  
and forgetting:  
n == 2
```

Correct:

```
if n <= 2:  
    return n
```

Mistake 2

Starting loop from wrong index.

Wrong:

```
for i in range(2, n+1)
```

May produce incorrect results.

Mistake 3

Updating variables in wrong order.

Wrong:

```
first = second
```

```
second = first + second
```

This uses modified values.

Edge Cases

Input t	Output t
1	1
2	2
3	3
4	5
5	8

12. Detailed Dry Run (Step-by-Step Table)

Input:

```
n = 5
```

Initial:

```
first = 1
```

```
second = 2
```



i	current	first	second
3	3	2	3
4	5	3	5
5	8	5	8

Return:

8

DP Array Visualization

$n = 5$

$dp[1] = 1$

$dp[2] = 2$

$dp[3] = dp[2] + dp[1]$

$= 2 + 1$

$= 3$

$dp[4] = dp[3] + dp[2]$

$= 3 + 2$

$= 5$

$dp[5] = dp[4] + dp[3]$

$= 5 + 3$

$= 8$

13. Common Use Cases (Real-Life / Interview)

Route Counting

How many ways can a robot move to a destination?

Path Planning

Navigation systems.

Resource Scheduling

Different ways to complete tasks.



Dynamic Programming Foundation

This problem introduces:

State

Transition

Base Cases

Optimization

which are core DP concepts.

14. Common Traps (Important!)

Trap 1

Using plain recursion.

Example:

$\text{ways}(n-1) + \text{ways}(n-2)$

Looks simple but becomes extremely slow.

Trap 2

Not identifying Fibonacci pattern.

Once recognized:

DP becomes easy.

Trap 3

Building huge recursion trees unnecessarily.

For:

$n = 45$

Recursive solution can perform millions of calls.

15. Builds To (Related LeetCode Problems)

Dynamic Programming Roadmap:

LC 70 - Climbing Stairs



LC 746 - Min Cost Climbing Stairs



LC 198 - House Robber



LC 213 - House Robber II





LC 322 - Coin Change



LC 518 - Coin Change II

These are classic DP interview questions.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Pure Recursion	$O(2^n)$	$O(n)$	Poor
Memoization	$O(n)$	$O(n)$	Good
DP Array	$O(n)$	$O(n)$	Good
Iterative DP	$O(n)$	$O(1)$	Best
Fibonacci Matrix	$O(\log n)$	$O(1)$	Advanced

Approach 1: Recursion

```
def climb(n):
```

```
    if n <= 2:
```

```
        return n
```

```
    return climb(n-1) + climb(n-2)
```

Simple but very slow.

Approach 2: Memoization

Store answers.

Already computed?

Use stored value.

Much faster.

Approach 3: Iterative DP (Chosen Solution)

Track only previous two answers.

Best interview solution.

17. Why This Solution Works (Short Intuition)

To reach stair n , the final move must come from:

stair $n-1$

or



stair $n-2$

Therefore:

$$f(n) = f(n-1) + f(n-2)$$

This recurrence generates the number of ways to reach each stair, making the problem identical to the Fibonacci sequence.

18. Variations / Follow-Up Questions

Follow-Up 1

What if you can climb:

1 step

2 steps

3 steps

How would the recurrence change?

Follow-Up 2

What if some stairs are broken?

Example:

Broken stairs = [3, 7]

Cannot land on them.

Follow-Up 3

Minimum cost to reach the top.

Related Problem:

Introduction to Algorithms style DP extension.

Follow-Up 4

Count ways if each stair has a weight limit.

Adds constraints to the state transition.

Interview Takeaway

When you see:

Number of ways

Reach a target

Choose between two previous states

immediately think:

Dynamic Programming

Fibonacci Pattern



Climbing Stairs is one of the most important beginner DP problems because it teaches the core idea:

Current Answer

=

Combination of Previous Answers